🎯 AX/DX 직무 실무 개념 총정리 — 내 프로젝트로 암기하기

> ***이 문서의 목적:*** *내가 만든* `Kids_Feedback` *프로젝트를 예시로, AX/DX 직무에서* ***말하고 이해할 수 있어야 하는 개념****을 한 장에 정리한다.* ***암기법:*** *각 개념은* `[한 줄 정의] → [왜 쓰나] → [내 프로젝트 어디에] → [면접 한마디]` *4단 구조.* ***면접 한마디만 30초에 말할 수 있으면 그 개념은 통과.***

---

0. AX/DX가 뭔데? — 그리고 내 프로젝트가 왜 그 자체인가

용어	뜻	핵심
DX (Digital Transformation)	사람이 손으로 하던 일을 **데이터·시스템으로** 바꾸기	"수기 → 디지털"
AX (AI Transformation)	그 시스템 안에 **AI를 심어** 일하는 방식 자체를 바꾸기	"자동화 → 지능화"

내 프로젝트가 정확히 AX/DX 사례다:

```
[기존/아날로그]  강사가 수업 끝나고 아이마다 손으로 피드백 작성 (반복 · 시간 · 편차)
      | DX = 데이터화
      ▼
[디지털]        아이 프로필 · 수업 기록을 DB에 저장, API로 관리
      | AX = AI 심기
      ▼
[지능화]        LLM이 아이 성향 반영해 피드백 자동 생성 + RAG로 우리 지도원칙 반영
```

👉 **면접 한마디:** *"현장(줄넘기 보조강사)의 반복 업무를 DB·API로 디지털화하고(DX), 그 위에 LLM과 RAG를 얹어 아이별 맞춤 피드백을 자동 생성하도록(AX) 만든 프로젝트입니다."*

AX/DX 담당자에게 요구되는 핵심 역량 3가지:

1. **현장 문제를 기술 구조로 번역** — "이 반복 업무를 어떤 부품으로 자동화할까"
2. **풀스택 이해(깊이보다 연결)** — API·DB·AI·RAG가 어떻게 맞물리는지 말할 수 있어야
3. **AI를 실제 업무에 안전하게 붙이기** — 프롬프트 규칙, 검토 단계, 개인정보 처리

---

1. 큰 그림 — 5부품 아키텍처 (⭐한 장 암기⭐)

> *AX/DX에서 제일 중요한 건 **"부품들이 어떻게 연결되나"**를 그림으로 말하는 것.*

```
[사용자/강사]
      |  화면에서 입력 · 클릭
      ▼
┌──────────────────────┐   ① Streamlit  = 화면 (프론트엔드)          ※ 아직 미구현
|  프론트엔드  |
└──────────────────────┘
      |  HTTP 요청 (JSON)
      ▼
```

```
┌──────────────────────┐    ② FastAPI    = 서버/API (요청 받아 처리)   main.py, routers/
│   백엔드      │    ↕ 부품들을 지휘하는 중심 "웨이터"
└──────────────────────┘
   ↙    ↓    ↘
┌──────────┐  ┌──────────┐  ┌──────────┐
│③ DB   │  │④ LLM  │  │⑤ RAG  │
│PostgreSQL│  │ Gemini │  │ 벡터검색 │
│창고    │  │ 요리사  │  │ 레시피북 │
└──────────┘  └──────────┘  └──────────┘
models    llm_service   rag/
```

#	부품	실무 명칭	비유	내 파일
①	화면	프론트엔드(Frontend)	식당 홀	(Streamlit, 예정)
②	서버	백엔드(Backend)/API	웨이터	`main.py`, `routers/`
③	데이터	데이터베이스(DB)	냉장고·장부	`models.py`, `database.py`
④	AI 생성	LLM	요리사	`llm_service.py`
⑤	AI 검색	RAG	요리사 옆 레시피북	`rag/`

⭐ **암기 문장:** *"프론트가 요청하면 백엔드가 받아, DB에서 데이터 꺼내고, RAG로 자료 찾아, 그걸 LLM에 줘서 답을 만들고, DB에 저장한 뒤 프론트에 돌려준다."*

---

2. 필수 개념 사전 — 영역별 (실무 용어 + 내 코드)

🟦 A. 백엔드 / API 영역

A-1. API (Application Programming Interface)

- **정의:** 프로그램끼리 **정해진 규칙으로 요청·응답을 주고받는 창구.**
- **왜:** 화면(프론트)과 서버(백엔드)를 **분리**하려고. 창구만 지키면 서로 안 건드리고 각자 개발.
- **내 프로젝트:** `POST /children` (아이 등록), `POST /feedbacks/generate` (피드백 생성) 같은 창구들.
- **면접 한마디:** *"기능을 URL 창구로 열어두면, 화면이든 다른 시스템이든 그 창구로 요청만 하면 됩니다."*

A-2. REST API + HTTP 메서드 (⭐빈출⭐)

- **정의:** 자원(데이터)을 **URL로 표현**하고, **동작은 HTTP 메서드로** 구분하는 API 설계 방식.
- **4대 메서드 = CRUD:**

메서드	뜻	CRUD	내 코드 예시
POST	생성	Create	`POST /children` 아이 등록
GET	조회	Read	`GET /children/{id}` 아이 조회
PATCH	부분수정	Update	`PATCH /children/{id}` 특징 수정
DELETE	삭제	Delete	(미구현)

- **CRUD** = Create/Read/Update/Delete. **모든 데이터 관리의 기본 4동작.**

- **면접 한마디:** *"REST는 URL로 자원을, HTTP 메서드로 동작을 나타냅니다. 생성은 POST, 조회는 GET, 수정은 PATCH, 삭제는 DELETE입니다."*

A-3. 엔드포인트 / 라우터 / prefix

- **엔드포인트(Endpoint):** 요청을 받는 **주소 한 개** (예: `/feedbacks/generate`).
- **라우터(Router):** 관련 엔드포인트를 **묶어놓은 파일**. 내 프로젝트는 `children_router`, `lesson_records_router`, `feedbacks_router` 로 기능별 분리.
- **prefix:** 라우터 공통 앞주소. `prefix="/feedbacks"` + `@router.post("/generate")` = 실제 주소 `/feedbacks/generate`.
- **면접 한마디:** *"기능별로 라우터를 나누면 유지보수가 쉽습니다. 아이 관련은 children_router로 묶었습니다."*

A-4. FastAPI + Swagger

- **FastAPI:** 파이썬 **웹 백엔드 프레임워크**. API를 빠르게 만들고, **자동으로 문서(Swagger)**를 만들어줌.
- **Swagger(`/docs`):** 코드를 짜면 자동 생성되는 **API 테스트 화면**. 프론트 없이도 버튼으로 API를 테스트. (내가 지금 Streamlit 대신 Swagger로 테스트하는 그거)
- **면접 한마디:** *"FastAPI는 타입 힌트 기반이라 자동 검증과 Swagger 문서가 딸려와서 개발·테스트가 빠릅니다."*

🟩 B. 데이터 / DB 영역

B-1. 데이터베이스(DB) + PostgreSQL

- **정의:** 데이터를 **구조적으로 저장·조회**하는 창고. **PostgreSQL** = 널리 쓰는 오픈소스 관계형 DB.
- **관계형(RDB):** 데이터를 **표(테이블)** 로 저장하고, 표끼리 **관계**로 연결.
- **내 프로젝트:** `children`, `lesson_records`, `feedbacks` 3개 테이블.
- **면접 한마디:** *"관계형 DB는 데이터를 표로 저장하고 표 사이를 관계로 잇습니다. 아이-수업기록-피드백을 관계로 연결했습니다."*

B-2. 테이블 · 컬럼 · PK · FK (⭐빈출⭐)

- **테이블(Table):** 한 종류의 데이터 표 (예: `children`).
- **컬럼(Column):** 표의 열/속성 (예: `name`, `age`, `personality`).
- **PK(Primary Key, 기본키):** 각 행을 **유일하게** 구분하는 번호. 내 코드: `id = Column(Integer, primary_key=True)`.
- **FK(Foreign Key, 외래키): 다른 표를 가리키는** 컬럼. 내 코드: `child_id = Column(Integer, ForeignKey("children.id"))` → "이 수업기록은 몇 번 아이 것".
- **면접 한마디:** *"PK는 각 행의 고유 번호, FK는 다른 표를 참조하는 연결고리입니다. lesson_records의 child_id가 children을 가리키는 FK입니다."*

B-3. 관계(Relationship) 1:N + ERD

- **1:N(일대다):** 아이 1명 ↔ 수업기록 여러 개. **가장 흔한 관계.**
- **ERD(Entity Relationship Diagram):** 테이블과 관계를 그린 **설계도**.

```
children (1) ──< (N) lesson_records (1) ──< (N) feedbacks
   아이 한 명이      수업기록 여러 개,  기록 하나에   피드백 여러 개
```

- **내 코드:** `relationship("LessonRecord", back_populates="child")` — 아이에서 기록들을 바로 꺼내 쓰게 연결.
- **면접 한마디:** *"아이 한 명에 수업기록이 여러 개인 1:N 구조입니다. ERD로 세 테이블 관계를 먼저 설계했습니다."*

B-4. ORM (SQLAlchemy) (⭐중요⭐)

- **정의: 파이썬 객체(class) ↔ DB 테이블**을 자동으로 매핑해주는 도구. SQL을 직접 안 쓰고 파이썬 코드로 DB 조작.
- **왜:** SQL 문자열 대신 `db.query(Child).filter(...)` 처럼 **파이썬답게** 쓰고, 실수·보안(SQL Injection) 위험도 줄임.
- **내 코드:**

```
# SQL:  SELECT * FROM lesson_records WHERE child_id=3 ORDER BY lesson_date DESC LIMIT 5
db.query(LessonRecord).filter(LessonRecord.child_id == child_id)₩
  .order_by(desc(LessonRecord.lesson_date)).limit(5).all()
```

- **면접 한마디:** *"ORM은 파이썬 객체와 DB 테이블을 매핑해, SQL을 직접 안 쓰고도 안전하게 DB를 다루게 해줍니다. SQLAlchemy를 썼습니다."*

B-5. 세션 · 커밋 · 트랜잭션

- **세션(Session):** DB와의 **작업 창구 하나**. `get_db()` 가 열고(`yield`) 끝나면 닫음(`close`).
- **저장 3단계:** `db.add()` (담기) → `db.commit()` (확정 저장) → `db.refresh()` (저장된 최신값 다시 읽기).
- **트랜잭션:** "**전부 성공 or 전부 취소**" 단위. commit 전엔 취소 가능.
- **면접 한마디:** *"add로 담고 commit으로 확정합니다. commit 전이면 롤백할 수 있어 데이터 일관성이 지켜집니다."*

🟧 C. 데이터 검증 / 스키마 영역

C-1. 스키마 + Pydantic (⭐중요⭐)

- **정의:** 주고받는 데이터의 **모양·타입을 정의하고 자동 검증**하는 것. FastAPI는 **Pydantic**으로 함.
- **왜:** 잘못된 데이터(문자열 자리에 숫자 등)를 **서버 들어오는 입구에서 막음** → 422 에러로 안내.
- **입력용 vs 응답용 분리:** `FeedbackGenerate` (입력: `lesson_record_id` 만) vs `FeedbackResponse` (응답: 전체 필드). **받는 것과 주는 것을 분리**하는 게 실무 관례.
- `from_attributes=True` : DB 객체(ORM)를 그대로 응답 모델로 변환 허용.
- **면접 한마디:** *"Pydantic 스키마로 입구에서 데이터 타입을 자동 검증합니다. 입력용과 응답용 스키마를 분리해 안전성을 높였습니다."*

🟥 D. LLM / 생성형 AI 영역 (AX의 핵심)

D-1. LLM (Large Language Model)

- **정의:** 방대한 텍스트로 학습해 **다음 단어를 예측**하며 글을 생성하는 AI (Gemini, GPT, Claude 등).
- **한계: 우리 회사만의 지식은 모른다** + 사실을 지어낼 수 있다(환각/Hallucination). → 그래서 RAG가 필요.
- **내 프로젝트:** `gemini-2.5-flash` 로 피드백 문장 생성.
- **면접 한마디:** *"LLM은 자연스러운 글은 잘 만들지만 우리만의 지식은 모릅니다. 그 한계를 RAG로 보완했습니다."*

D-2. 프롬프트 엔지니어링 (⭐AX 필수⭐)

- **정의:** LLM에게 주는 **지시문(프롬프트)을 설계**해 원하는 출력을 얻는 기술.
- **좋은 프롬프트 요소 (내 `build_prompt` 에 다 있음):**
  1. **역할 부여:** "당신은 아동 줄넘기 코치입니다"
  2. **맥락 제공:** 아이 프로필 + 최근 기록
  3. **규칙 명시:** "외우기용 특징은 학부모용에 노출 금지", "다른 아이와 비교 금지"

4. **출력 형식 고정:** "아래 JSON 형식으로만 출력"

- **면접 한마디:** *"프롬프트에 역할·맥락·규칙·출력형식을 명확히 넣어야 일관된 결과가 나옵니다. 민감 정보 노출 금지 규칙도 프롬프트로 통제했습니다."*

D-3. 구조화 출력 (JSON 모드)

- **정의:** LLM 답을 **정해진 JSON 구조**로 받는 것. 시스템이 바로 파싱·저장하게.
- **내 코드:** `config={"response_mime_type": "application/json"}` → ```` ```json ```` 없이 순수 JSON. `json.loads()` 로 파이썬 딕셔너리로 변환 후 DB 저장.
- **면접 한마디:** *"LLM 출력을 JSON으로 고정하면 시스템이 바로 파싱해 DB에 저장할 수 있습니다."*

D-4. API 키 · 환경변수 · 보안

- **API 키:** 유료 AI 서비스를 쓰는 **비밀 열쇠**. 코드에 직접 쓰면 유출 위험.
- `.env` + **환경변수:** 비밀값을 **코드 밖 파일**에 두고 불러옴. `load_dotenv()` → `os.getenv("GEMINI_API_KEY")` . `.env` 는 **git에 올리지 않음**(.gitignore).
- **면접 한마디:** *"API 키 같은 비밀값은 .env에 분리하고 git에서 제외해 유출을 막습니다."*

🟪 E. RAG / 벡터 검색 영역 (⭐ ⭐AX 최빈출⭐ ⭐)

E-1. RAG (Retrieval-Augmented Generation)

- **정의:** LLM이 답하기 **직전에 관련 자료를 검색(Retrieval)해 프롬프트에 넣어(Augmented) 답을 생성(Generation)**하는 구조.
- **왜:** LLM이 모르는 **우리만의 지식**(사내 규정, 제품 매뉴얼 등)을 반영 + **환각 감소** + 매번 프롬프트에 길게 안 넣어도 됨(확장성).
- **내 프로젝트:** "위축형 아이는 결과 말고 시도를 칭찬한다" 같은 **우리 지도원칙**을 문서화 → 검색해서 피드백에 반영.
- **면접 한마디:** *"RAG는 답 직전에 우리 자료를 검색해 프롬프트에 넣는 구조입니다. LLM이 모르는 사내 지식을 반영하고 환각을 줄입니다."*

E-2. RAG 4단계 파이프라인 (⭐순서 암기⭐)

```
[준비 단계 - 한 번만]
① 청킹(Chunking)   문서 → 의미 단위 조각      (내 코드: .split("##"))
② 임베딩(Embedding) 조각 → 숫자 벡터(3072개)   (gemini-embedding-001)
③ 저장(Vector Store) 벡터를 검색 가능하게 보관  (vectors.json)

[검색 단계 - 질문 올 때마다]
④ 검색(Retrieval)  질문도 벡터화 → 가장 가까운 조각 찾기 (cos_sim)

[생성 - 이어서]
⑤ 생성(Generation) 찾은 조각을 프롬프트에 넣어 LLM 호출
```

- **면접 한마디:** *"문서를 잘라서(청킹) 숫자로 바꿔(임베딩) 저장해두고, 질문이 오면 뜻으로 가장 가까운 조각을 찾아 LLM에 넣습니다."*

E-3. 청킹 (Chunking)

- **정의:** 긴 문서를 **검색·투입하기 좋은 조각**으로 자르기.
- **기준(중요):** 크기가 아니라 **의미**. 내 기준 = "**질문 1개 = 대답 1개**". 조각만 떼어 읽어도 말이 돼야 함(독립성).
- **왜 중요:** 잘못 자르면 검색 정확도가 통째로 무너짐. **RAG 품질의 8할이 청킹.**

- **면접 한마디:** *"청킹은 크기가 아니라 의미 단위로 잘라야 합니다. 저는 질문-답변 한 쌍을 한 조각으로 잡았습니다."*

E-4. 임베딩 (Embedding) + 벡터 (Vector)

- **정의: 글자를 숫자 목록(벡터)으로 변환**. 뜻이 비슷하면 숫자도 가깝게 나옴.
- **핵심:** 컴퓨터는 "위축된 아이"와 "소심한 애"가 비슷한 걸 **글자로는** 모름 → 숫자로 바꿔 **계산으로** 비교.
- **차원(Dimension):** 벡터의 칸 수. 내 모델은 **3072차원**. 글 길이와 무관하게 **항상 고정**(고정돼야 비교 가능).
- **면접 한마디:** *"임베딩은 글을 벡터로 바꿔, 글자가 안 겹쳐도 의미로 비교할 수 있게 합니다."*

E-5. 벡터 검색 + 코사인 유사도 (⭐빈출⭐)

- **코사인 유사도(Cosine Similarity):** 두 벡터의 **방향(각도)**이 얼마나 같은지. **1에 가까울수록 비슷, 0이면 무관.**
- **왜 각도(방향)냐:** 글 길이(크기)에 안 휘둘리고 **뜻**만 공정하게 비교하려고.
- **의미 검색(Semantic Search):** 키워드가 아니라 **뜻**으로 찾기. "소심한 애" 질문에 "위축형" 조각이 걸리는 이유.
- **내 코드:** `cos_sim(q_vector, d["vector"])` 8개 비교 → `scores.index(max(scores))` 로 1등 선택.
- **면접 한마디:** *"질문과 각 조각의 코사인 유사도를 계산해 가장 높은 조각을 고릅니다. 키워드가 아니라 의미로 검색합니다."*

E-6. 벡터 스토어 (Vector Store)

- **정의:** 벡터를 저장하고 **빠르게 유사 검색**하게 해주는 저장소.
- **내 프로젝트:** 지금은 `vectors.json` 파일(학습용). **실무에선** 전용 벡터 DB 사용 — **Chroma, Pinecone, FAISS, pgvector**(PostgreSQL 확장) 등.
- **왜 저장하나:** 임베딩은 **API 호출(돈·시간)**. 한 번 만들어 저장하면 다음부턴 읽기만.
- **면접 한마디:** *"학습용으로 JSON에 저장했지만, 실무에선 Chroma나 pgvector 같은 벡터 DB로 확장할 수 있습니다."*

⚙️ F. 개발 운영 / 도구 영역 (실무 기본기)

개념	한 줄 정의	내 프로젝트	면접 한마디
가상환경 (venv)	프로젝트별 독립 파이썬 방	`.venv` (uv로 관리)	"프로젝트마다 라이브러리를 분리해 충돌을 막습니다"
패키지 매니저	라이브러리 설치·버전 고정	`uv`, `pyproject.toml`, `uv.lock`	"uv로 의존성을 잠가 재현 가능한 환경을 만듭니다"
Git / GitHub	버전 관리·협업	commit·push로 이력 관리	"작업 단위로 커밋해 되돌리기와 협업이 가능합니다"
.gitignore	git에 안 올릴 것 지정	`.env`, `.venv` 제외	"비밀값·환경은 저장소에서 제외합니다"
의존성 주입 (DI)	필요한 걸 밖에서 넣어줌	`db: Session = Depends(get_db)`	"DB 세션을 주입받아 테스트·교체가 쉽습니다"
예외 처리	에러를 안전하게 안내	`HTTPException(404, ...)`	"없는 데이터는 404로 안내해 서버가 죽지 않게 합니다"
환경 이슈	경로·인코딩·파이썬 버전	상대경로·cp949·venv 함정	(실무에서 가장 자주 만나는 벽)

---

3. 실무 관점 — 이 프로젝트를 AX/DX 사례로 설명하기

문제 → 해결 스토리 (면접 핵심)

```
문제:  강사가 아이마다 수업 후 피드백을 손으로 작성 → 반복 · 시간 · 품질 편차
분석:  아이별 특성 관리가 안 되고, 개수 위주 피드백의 한계
해결:  ① 아이 프로필 · 수업기록을 DB화 (DX)
       ② LLM으로 성향 반영 피드백 자동 생성 (AX)
       ③ RAG로 우리 학원 지도원칙을 일관되게 반영 (AX 고도화)
효과:  작성 시간 단축 + 아이별 맞춤성 + 지도 품질 표준화
```

AX/DX 담당자가 이 프로젝트에서 신경 쓴 "실무 포인트"

1. **역할 분리 아키텍처** — 화면/서버/DB/AI/RAG를 파일로 분리 → 유지보수·교체 용이
2. **AI 안전장치** — 민감정보 노출 금지 프롬프트 규칙 + "강사 검토 후 사용"(사람이 최종 확인) + 가명 데이터
3. **확장성 설계** — 벡터스토어를 JSON→pgvector로, 단일흐름을 나중에 LangGraph로 확장 가능하게 여지
4. **왜 안 썼나까지 설명** — "이번 MVP엔 멀티에이전트/LangGraph는 과함" → **범위 관리** 능력 어필

자주 나오는 "왜?" 질문 대비

질문	핵심 답
왜 RAG를 썼나?	지도원칙을 매번 프롬프트에 넣기보다 **문서화+검색**이 일관·확장 가능
왜 LLM만으로 안 되나?	우리만의 지식 모름 + 환각 → RAG로 근거 제공
개인정보는?	가명 샘플 + 강사 검토 후 사용 + 진단·비교 표현 금지 규칙
왜 멀티에이전트 안 썼나?	MVP 범위엔 과함, 단일 흐름부터 안정화 후 확장

---

4. ⭐30초 자기소개 스크립트 (통째 암기)⭐

> *"줄넘기 보조강사 경험에서 출발한 프로젝트입니다. 강사가 아이마다 손으로 쓰던 피드백 업무를, 아이 프로필과 수업 기록을 **PostgreSQL·FastAPI로 디지털화(DX)**하고, 그 위에 **Gemini LLM으로 성향 맞춤 피드백을 자동 생성(AX)**했습니다. 핵심은 **RAG**인데요, 우리 학원만의 지도원칙을 문서로 만들어 **청킹·임베딩·벡터검색**으로 찾아 프롬프트에 넣어, LLM이 일반론이 아니라 **우리 방식대로** 답하게 했습니다. 민감정보 노출 금지 규칙과 강사 검토 단계로 안전성도 챙겼습니다."*

이 안에 든 키워드 15개 (다 설명 가능해야 함): DX / AX / PostgreSQL / FastAPI / API / DB / ORM / LLM / 프롬프트 엔지니어링 / RAG / 청킹 / 임베딩 / 벡터검색 / 코사인 유사도 / 구조화 출력(JSON)

---

5. 🧠 셀프 암기 체크리스트 (안 보고 30초 설명)

> *각 항목을 **안 보고 → 내 말로 30초 → 이 문서로 확인**. 3번 통과하면 그 개념 졸업.*

큰 그림

- [ ] AX와 DX의 차이는? 내 프로젝트가 왜 둘 다인가?
- [ ] 5부품(프론트/백엔드/DB/LLM/RAG)을 그림으로 그리고 요청 흐름 설명하기

백엔드/API

- [ ] REST에서 CRUD ↔ HTTP 메서드(POST/GET/PATCH/DELETE) 짝짓기
- [ ] API를 왜 쓰나 (프론트-백 분리) / 엔드포인트·라우터·prefix 관계
- [ ] FastAPI와 Swagger가 뭔가

데이터

- [ ] PK와 FK 차이, 내 프로젝트의 FK 예시
- [ ] 1:N 관계와 ERD를 그려 설명 (children-lesson_records-feedbacks)
- [ ] ORM이 뭐고 왜 쓰나 (SQLAlchemy)
- [ ] add→commit→refresh 3단계, 트랜잭션

검증

- [ ] 스키마(Pydantic)가 왜 필요한가 / 입력용·응답용 분리 이유

LLM/AI

- [ ] LLM의 한계 2가지 (모름 + 환각)
- [ ] 좋은 프롬프트 4요소 (역할/맥락/규칙/출력형식)
- [ ] JSON 구조화 출력을 왜 쓰나
- [ ] API 키를 .env로 빼는 이유

RAG (⭐제일 중요⭐)

- [ ] RAG 한 문장 정의 (Retrieval-Augmented Generation)
- [ ] RAG 4단계 순서 (청킹→임베딩→저장→검색→생성)
- [ ] 청킹 기준은 크기가 아니라 ___
- [ ] 임베딩이 뭐고 벡터 차원이 왜 고정돼야 하나
- [ ] 코사인 유사도로 뭘 재나, 왜 각도(방향)인가
- [ ] 벡터스토어 실무 예시 3개 (Chroma/pgvector/FAISS...)

운영

- [ ] venv를 왜 쓰나 / 의존성 주입(Depends)이 뭔가 / 예외처리(404)를 왜 하나

📌 다음 학습 방향 (AX/DX 심화)

- **LangChain / LangGraph** — RAG·에이전트 파이프라인 프레임워크 (내가 "안 썼다"고 말한 그것)
- **벡터 DB 실물** — Chroma나 pgvector로 `vectors.json` 교체해보기
- **평가(Evaluation)** — RAG 답이 정확한지 측정하는 법 (RAGAS 등)
- **에이전트(Agent)** — LLM이 도구를 스스로 골라 쓰는 구조
- **MLOps/LLMOps 기초** — 배포·모니터링·비용관리

> *지금 이 프로젝트 하나로 위 심화의 **뿌리 개념(API·DB·LLM·RAG)**은 전부 손에 넣은 상태다. 심화는 이 뿌리에 가지를 치는 것뿐.*